

Ingresso/uscita (note)

AESO, Corso A e B, Anno Accademico 2025–26

M. Danelutto

23 febbraio 2026

Indice

1	Ingresso uscita	5
1.1	Bus	5
1.1.1	Arbitri	7
1.1.2	Protocolli sincroni e asincroni	9
1.2	Memory mapped I/O	10
1.3	DMA	16
1.3.1	Dispositivi DMA	17
1.3.2	Sull'utilizzo del DMA	18
1.4	Interruzioni	19
1.5	Utilizzo di un dispositivo di I/O	25
1.5.1	Memory mapped I/O e polling	25
1.5.2	Interrupt-driven I/O	26
1.6	Ingresso uscita: divisione delle responsabilità	28

Capitolo 1

Ingresso uscita

In questo capitolo delle note vediamo tre cose in particolare, che sul libro di testo sono solo accennate oppure non sono neppure trattate: memory mapped I/O, DMA e trattamento delle interruzioni. Tutte e tre hanno un ruolo fondamentale nella gestione dell'ingresso/uscita.

1.1 Bus

Prima di cominciare a vedere i meccanismi che servono per l'implementazione del sottosistema di ingresso uscita, vediamo cosa sono i *bus*.

Un bus è un meccanismo per comunicazioni che coinvolgono diverse unità, ovvero che mette in comunicazione fra di loro n unità, anche diverse fra di loro e con ruoli diversi nelle comunicazioni. Il bus permette sia di realizzare comunicazioni punto a punto fra le unità connesse, che di realizzare comunicazioni *collettive*, ovvero comunicazioni con una sorgente e destinazioni multiple. Un esempio di comunicazione collettiva è il *broadcast* che manda un messaggio da una unità sorgente a *tutte* le unità affacciate sul bus di comunicazione contemporaneamente.

Fin'ora abbiamo visto sempre e solo collegamenti dedicati. Per esempio, il collegamento fra la memoria e il processore è un collegamento dedicato. Ha linee per la trasmissione di operazione, indirizzo ed eventuale dato da scrivere verso che vengono scritte dal processore e linee per la trasmissione dei dati letti dalla memoria ed eventuali esiti dell'operazione di lettura o scrittura che vengono scritte dalla memoria e lette dal processore. Un altro esempio di collegamento dedicato, nel processore single cycle, è quello che collega il PC alla memoria istruzioni (vedi fig. 7.13 del libro Harris&Harris, di cui riportiamo la parte rilevante in Fig. 1.1). In questo caso il collegamento è una linea da 32 bit in uscita dal registro PC e in ingresso sull'ingresso indirizzi (A, in figura) della memoria istruzioni.

Un bus è formato da un insieme di linee di tipo e cardinalità diversi:

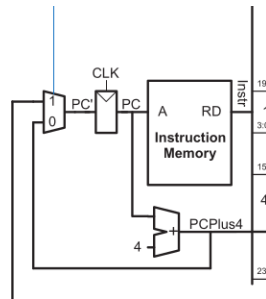


Figura 1.1: Collegamento PC–MI nel processore single cycle

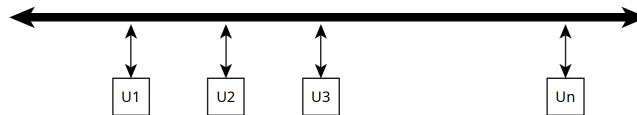


Figura 1.2: Rappresentazione grafica di un Bus con le unità collegate

- linee dati, che permettono di comunicare bit/byte/parole da una unità sorgente a una unità destinataria;
- linee indirizzo, che permettono di indicare l'indirizzo delle unità destinazione e/o sorgenti delle comunicazioni;
- linee controllo, che portano i segnali di controllo necessari all'implementazione del (dei) protocollo(i) di comunicazione che opera(n) sul bus.

Un bus di comunicazione lo rappresenteremo come uno o più linee parallele che terminano con una freccia su entrambi i lati, e le unità che hanno accesso al bus le rappresenteremo come unità con un collegamento dedicato alle linee del bus (vedi Fig. 1.2).

Un esempio di comunicazione su bus potrebbe essere il seguente. Immaginiamo un bus che abbia 8 linee di indirizzo unità (indirizzo da 8 bit), 16 linee di indirizzo memoria, 32 linee dati e una linea di controllo che dice se leggere o scrivere (vale 0 se indichiamo una operazione di lettura e 1 se la indichiamo di scrittura). Immaginiamo anche che le unità che sono collegate al bus, siano fatte in modo da avere ciascuna una unità interna di memoria. L'unità U_1 potrebbe scrivere sulle linee indirizzo il valore 4, sulle linee indirizzo memoria il valore 1024, sulle linee dati la parola 0xff00ff00 e infine mettere a 1 la linea di controllo, con l'intento di comunicare all'unità U_4 la richiesta di scrivere nella sua memoria interna, all'indirizzo 1024 la parola 0xff00ff00.

Potremmo anche immaginare che il protocollo di comunicazione utilizzato preveda una operazione di *broadcast* mediante l'utilizzo dell'indirizzo di unità 0xff. In questo caso una situazione come quella appena discussa, ma con l'unità U_1 che scrive 0xff sulle linee indirizzo unità, farebbe sì che *tutte* le unità affacciate sul bus (compresa U_1) eseguano la scrittura della parola 0xff00ff00 all'indirizzo 1024 della loro memoria interna.

Essendo un bus un mezzo di comunicazione condiviso fra diverse unità abbiamo però un problema. Come fare a fare in modo che il bus non venga utilizzato da più unità contemporaneamente, magari per operazioni diverse? Questa situazione sarebbe ingestibile. Anche nella migliore delle ipotesi (linee del bus che si comportano come wired OR), unità diverse che scrivono valori diversi sulle linee del bus porterebbero a vedere segnali che non sono validi, o che comunque non sono i segnali che ciascuna delle unità intendeva utilizzare sulle linee del bus.

Per la gestione dei bus si ricorre quindi all'utilizzo di unità di *arbitraggio* che accettino richieste di utilizzo del bus e ne assegnino l'uso *esclusivo* a una delle unità che ne hanno fatto richiesta secondo una *politica di gestione*, fino a quando l'unità non termina la propria operazione sul bus e lo *rilascia* per l'utilizzo da parte delle altre unità che ne fanno richiesta.

L'utilizzo del bus diventa quindi un algoritmo a due fasi:

- l'unità che intende accedere al bus fa una richiesta all'arbitro;
- se e quando ottiene l'accesso al bus, lo utilizza per l'operazione che intende fare e successivamente comunica all'arbitro che l'operazione è terminata.

1.1.1 Arbitri

Vediamo tre tipi di arbitri che possono essere utilizzati per controllare l'accesso ad un bus.

Arbitro a richieste indipendenti

Questo tipo di arbitro controlla n unità. Per ciascuna unità ha due linee da 1 bit: una linea in ingresso, di richiesta (r_i per l'unità i), e una linea in uscita (a_i

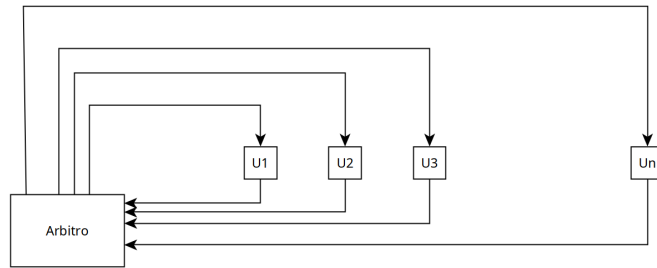


Figura 1.3: Arbitro a richieste indipendenti

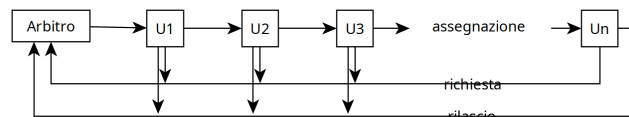


Figura 1.4: Arbitro Daisy Chaining

per l'unità i), di assegnazione (vedi Fig. 1.3). Le unità che intendono richiedere l'accesso alla risorsa controllata dall'arbitro mettono a uno la propria linea di richiesta. L'arbitro, secondo la sua politica di assegnazione, sceglie a quale delle unità che hanno fatto richiesta assegnare la risorsa e mette a 1 la linea di assegnazione per quella unità. L'unità che avendo fatto richiesta vede la linea di assegnazione a 1, accede al bus, fa l'operazione e successivamente mette a 0 la linea di richiesta. L'arbitro corrispondentemente mette a 0 la linea di assegnazione e ricomincia a valutare le eventuali altre richieste pendenti.

Arbitro daisy chaining

Un arbitro di questo tipo ha due linee in ingresso da un bit e una linea in uscita, anch'essa di un bit. Le linee di ingresso sono una linea di richiesta e una linea di rilascio. La linea in uscita è una linea di assegnazione dell'uso della risorsa (vedi Fig. 1.4). Le unità hanno accesso in scrittura ad entrambe le linee di richiesta e rilascio, che si comportano come wired OR. La linea di assegnazione è collegata alla prima unità. Ciascuna unità ha in ingresso la linea di assegnazione e in uscita un'altra linea che permette di passare il segnali di assegnazione all'unità successiva, come nello schema in Fig. 1.4.

L'unità che intende utilizzare la risorsa mette a 1 la linea di richiesta. Se la risorsa è libera, l'arbitro mette a 1 la linea di assegnazione. La prima unità vede arrivare il segnale di assegnazione a 1. Se aveva fatto richiesta, utilizza il bus e successivamente mette a 1 la linea di rilascio. Altrimenti passa il segnale

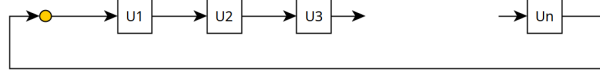


Figura 1.5: Arbitro Token Ring

di assegnazione alla prossima unità. Le altre unità si comportano in modo analogo. L'arbitro che ha messo a 1 il segnale di assegnazione attende che il segnale di rilascio sia 1 prima di procedere ad una altra assegnazione della risorsa.

Da notare che in questo tipo di arbitraggio la politica è fissa e la priorità è determinata dall'ordine in cui le unità sono collegate. Peraltro, aggiungere un'altra unità che volesse utilizzare la stessa risorsa non necessita di un cambiamento dell'arbitro, cosa che invece è necessaria per le richieste indipendenti, visto che ogni unità richiede collegamenti dedicati distinti con l'arbitro.

Arbitro token ring

L'arbitro token ring è completamente decentralizzato. Le unità che utilizzano l'arbitraggio hanno tutte un ingresso da 1 bit e un'uscita anch'essa da un bit che sono collegati a formare un anello (ring) come in Fig. 1.5. Sull'anello circola un token (bit 1). Ciascuna delle unità legge uno 0 in ingresso o un 1. 0 vuol dire che non ha il permesso di utilizzare la risorsa, 1 che invece ne ha diritto. Quando in ingresso l'unità U_i vede un 1, se intende utilizzare la risorsa, scrive 0 sull'uscita (mantiene lo zero che già c'era), utilizza la risorsa e, quando l'utilizzazione è terminata, scrive 1 sull'uscita, passando di fatto il token all'unità successiva.

Anche in questo caso, c'è un ordinamento fra le unità dettato dai collegamenti nelle posizioni dell'anello. Quando più unità vogliono utilizzare la risorsa, l'unità che si trova immediatamente dopo la posizione corrente del token ha precedenza sulle altre. Il fatto che il token circoli anche in assenza di utilizzo della risorsa garantisce una distribuzione equa delle possibilità di accesso a tutte le unità.

1.1.2 Protocolli sincroni e asincroni

I protocolli di utilizzo dei bus possono essere sincroni o asincroni.

Nel primo caso, si assume che le unità che accedono al bus abbiano di fatto un clock a comune e tutte le transazioni avvengono scandite dal tempo. Per esempio, una volta ottenuto l'accesso alla risorsa, avviene la scrittura dell'operazione sul bus (1τ) poi dopo un certo numero di cicli di clock (1 o più τ) si può osservare il risultato dell'operazione sul bus.

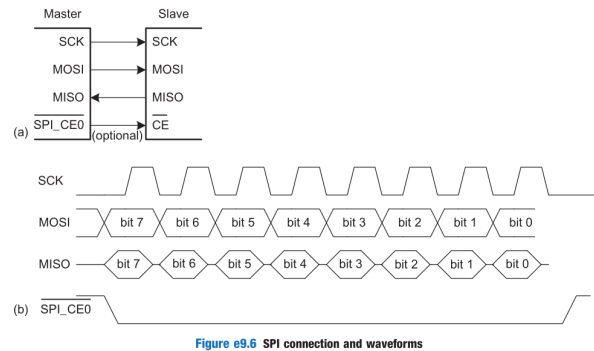


Figure e9.6 SPI connection and waveforms

Figura 1.6: Protocollo sincrono (SPI)

Un esempio di protocollo sincrono è quello della Fig. e9.6 del libro Harris&Harris che riportiamo in Fig. 1.6 per comodità, che rappresenta la trasmissione di bit nel protocollo SPI utilizzato nel mondo IoT. Il segnale di clock in questo caso scandisce il tempo. La trasmissione di un byte inizia quando il clock è basso e il segnale *SPI_CEO* va basso. Successivamente viene trasmesso un bit ogni volta che il clock va alto.

Nel secondo caso, opportuni segnali di controllo determinano le fasi del protocollo. Per esempio, l'unità che ha ottenuto l'uso del bus scrive le linee che servono per implementare l'operazione e successivamente attende che una o più altre linee segnalino che l'operazione è terminata e con quali esiti.

Ad esempio, un protocollo che preveda che una linea *RQ* del bus venga messa a 1 per iniziare un'operazione e che l'esito dell'operazione sia disponibile solo quando un'altra linea *END* viene messa a 1, indipendentemente da come siano i clock delle unità coinvolte, è un asincrono.

1.2 Memory mapped I/O

Per memory mapped I/O si intende la possibilità di accedere ed interagire con i dispositivi di ingresso uscita utilizzando le istruzioni messe a disposizione dal linguaggio assembler per interagire con la memoria, ovvero le istruzioni di load e store.

L'astrazione che utilizziamo per modellare un dispositivo di ingresso uscita è quella dell'unità firmware, ovvero un dispositivo hardware che quando viene acceso comincia ad eseguire un ciclo infinito nel quale attende che gli venga ordinato di eseguire un comando, lo esegue, ne riporta i risultati e ricomincia la prossima iterazione:

```
dispositivoI/O:
while(true) {
    read(comando, parametri);
```

```
results = exec(comando, parametri);
writeback(results);
}
```

Per essere più precisi, consideriamo che il dispositivo prenda in considerazione l'esecuzione del comando con i parametri solo quando una certa variabile (per esempio `go`) viene messa a `true`. Questo vuol dire che il dispositivo in realtà esegue un programma di controllo tipo:

```
dispositivoI/O:
while(true) {
    while(!(go));
    read(comando, parametri);
    results = exec(comando, parametri);
    writeback(results);
}
```

Consideriamo un dispositivo di input qual'è la tastiera. Il tipico comando che gli può essere impartito è quello di lettura di un carattere. L'esecuzione del comando comporta l'attesa dello schiacciamento di un tasto da parte dell'utilizzatore della tastiera. Il risultato da restituire, quando il tasto sia stato effettivamente premuto, è il codice del tasto che è stato schiacciato. Supponiamo per il momento che questa sia l'unica operazione che possa essere richiesta al dispositivo tastiera.

Il dispositivo disporrà tipicamente di alcuni registri che permettono di memorizzare comandi, parametri e risultati delle operazioni che sono in grado di eseguire. Nel caso della tastiera ci possiamo immaginare di avere:

- un registro da 1 bit che sia interpretato come ordine di lettura di un tasto: se il bit è a 0, non è richiesta alcuna lettura, se il bit è a 1, deve essere letto un carattere (questo di fatto è il `go`, visto che l'unico comando che può eseguire la nostra tastiera è leggere il prossimo carattere. Immaginiamo che venga rimesso a 0 una volta letto il carattere);
- un registro lungo abbastanza per contenere il numero di bit utilizzati per rappresentare uno qualunque dei tasti che possono essere premuti. Su una tastiera tradizionale, il codice relativo ad un tasto potrebbe corrispondere alle sue coordinate riga/colonna. Di norma ci sono 6 righe di tasti (compresi i tasti Fn) e una quindicina di colonne diverse (spesso non perfettamente verticali). Per rappresentare un tasto serviranno quindi ca 21 bit. Immaginiamo che il tasto possa dunque essere rappresentato su una configurazione da 32 bit, contando anche che servono altri bit per indicare se il tasto è stato premuto insieme a uno shift, control, alt, meta, fn ...

Con queste ipotesi, il ciclo di funzionamento del nostro dispositivo tastiera potrebbe essere:

```
while(true) {
    while(!go);
    if(ordinedilettura == 1) {
        ... attendi pressione tasto ...;
        codice = codice(tastoPremuto);
        0 -> ordainedilettura;
    }
}
```

La tecnica del memory mapped I/O permette di utilizzare le normali istruzioni di load e store (LDR e STR nel caso di ARM) per andare a scrivere e leggere i registri interni dell'unità. In pratica, supponiamo di avere a disposizione degli indirizzi codificati in modo da essere dirottati sull'unità di I/O invece che nella memoria. Per esempio, immaginiamo, per il momento, che i registri della nostra tastiera corrispondano agli indirizzi 1024 e 1025.

Un programma che voglia leggere un dato dalla tastiera dovrebbe quindi eseguire un codice tipo:

```
1      mov r0, #1024    @ indirizzo base del dispositivo
2      mov r1, #1       @ costante 1
3      str r1, [r0]     @ ordina la lettura di un tasto
4 wait: ldr r1, [r0]     @ controlla termina lettura
5      cmp r1, #1
6      beq wait        @ e in caso attendi
7      ldr r0, [r0, #4] @ carica codice del tasto letto
8      mov pc, lr       @ ritorno al chiamante
```

ATTENZIONE: questo modo di interagire con l'ingresso uscita è assolutamente inefficace: vengono eseguite istruzioni a vuoto (ciclo "wait") in attesa che l'utente schiacci un tasto. Un ritardo di un paio di secondi da parte dell'utente a schiacciare il tasto dopo che è stata fatta partire la routine di lettura implica, su una macchina con un clock da 1GHz, l'esecuzione di qualcosa come 2sec/3nsec istruzioni in attesa che venga letto il carattere! (vedi Sez. 1.4)

Ma come facciamo a fare in modo che le load e le store del codice abbiano effetto sul dispositivo e non siano interpretate come normali load e store sul sottosistema di memoria?

Il processore riconosce alcuni indirizzi come appartenenti allo spazio di I/O (è questo il vero e proprio concetto di memory mapped I/O) e, tramite la MMU, redirige le richieste relative a questi indirizzi sul bus che collega processore e dispositivi di I/O (vedi Fig. 1.7). In sostanza, lo spazio degli indirizzi di memoria viene visto come composto da due parti distinte:

- una parte degli indirizzi sono utilizzati per accedere la memoria principale;

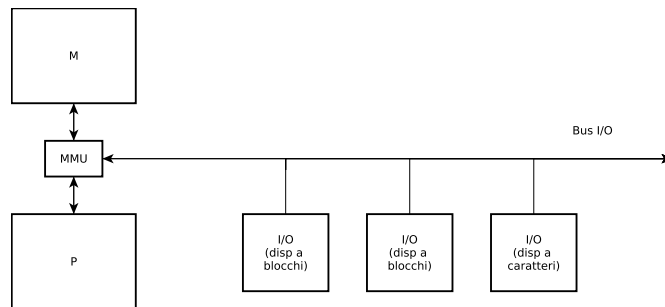


Figura 1.7: Bus I/O, con MMU che redirige i comandi LDR/STR sulla memoria principale o sul bus di I/O.

- una parte degli indirizzi per accedere le memorie interne dei dispositivi di I/O.

La Fig. 1.8 fa vedere la situazione tipica di ARMv7, che vede gli indirizzi dedicati all'I/O nella parte alta dello spazio di indirizzamento. Tutti i dispositivi di I/O vedono passare le operazioni, ma ognuno intercetta solo quelle che fanno riferimento agli indirizzi che gli sono stati assegnati. Per esempio, il nostro dispositivo tastiera controllerà quali sono gli indirizzi indicati nelle load e store che passano sul bus e tratterrà per se, ed eseguirà opportunamente, solo quelli relativi all'indirizzo 1024 e 1025. Lo schema semplificato che implementa questa soluzione è quello riportato nella Fig. e9.1 del libro¹.

Lo schema della Fig. e9.1 in realtà fa riferimento a un modello molto semplice in cui l'unità di ingresso uscita accede esclusivamente un registro in ingresso e di conseguenza produce un dato in uscita. Il dispositivo di I/O in oggetto riceverà tutti i dati relativi alle operazioni di lettura scrittura effettuate nello spazio di indirizzamento di ingresso uscita.

Per come lo abbiamo descritto, il memory mapped I/O fa in realtà corrispondere un certo insieme di indirizzi a indirizzi di una memoria interna alla unità di ingresso uscita. Vediamo come questo funziona con un semplice esempio.

Immaginiamo che gli indirizzi dedicati all'I/O siano gli indirizzi nella parte alta dello spazio di indirizzamento. Per esempio quelli che hanno i 2 bit più significativi dell'indirizzo a uno. In esadecimale, saranno gli indirizzi compresi fra 0xc0000000 e 0xffffffff. Con questa ipotesi, avremmo 1G indirizzi per la memoria delle unità di I/O e 3G indirizzi per la memoria vera e propria.

Supponiamo anche che un certo dispositivo di I/O abbia una memoria da 1K parole (quindi con indirizzi che vanno da 0x000 a 0x3ff).

In fase di configurazione del dispositivo (algoritmi *plug&play*) al dispositivo sarà stato assegnata una porzione degli indirizzi riservati all'ingresso uscita. Per semplicità, supponiamo che gli siano stati assegnati gli indirizzi da 0xC0000000 a 0xC00003ff.

¹Il capitolo 9 del libro, che parla dell'ingresso uscita, può essere scaricato dal sito web dell'editore in forma PDF

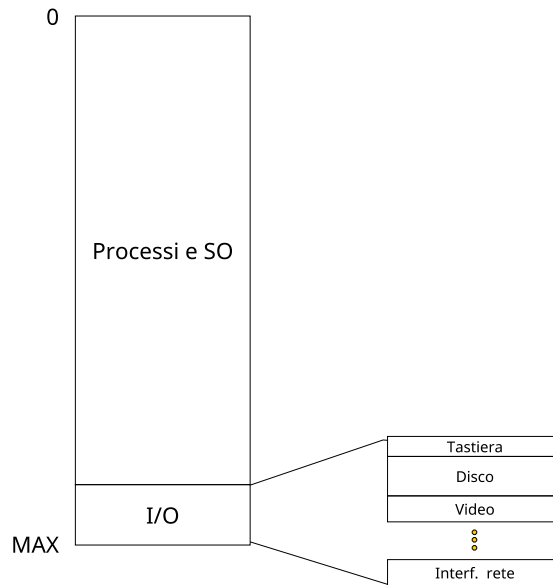


Figura 1.8: Spazio di indirizzamento in memoria (fisica)

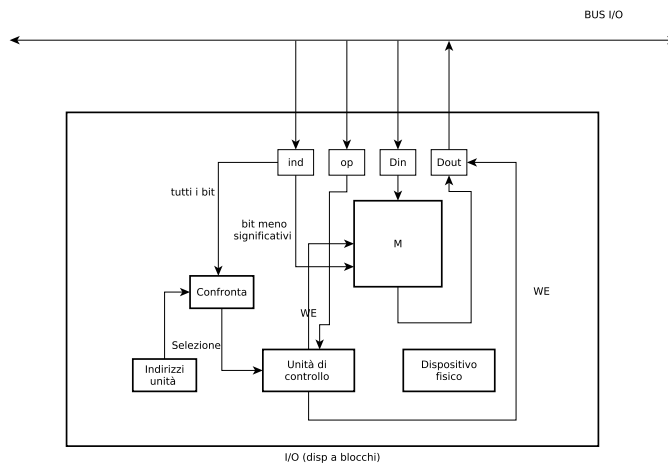


Figura 1.9: Unità di I/O

Come prima cosa, le operazioni di load e store eseguite dal processore verranno dirottate verso

- il sottosistema di memoria se l'AND dei bit IND[31:30] è 0 (combinazioni 10, 01 e 00)
- il sottosistema di I/O se l'AND degli stessi bit è pari a 1 (ovvero i due bit sono 11).

Quando l'unità di ingresso uscita vede passare un indirizzo, controlla che sia compreso nell'intervallo 0xC0000000-0xC00003ff. Se questa condizione è vera, utilizza gli ultimi bit come indirizzo della propria memoria interna di 1K parole ed esegue l'operazione richiesta dal processore (LDR o STR). Diversamente ignora l'operazione (la Fig. 1.9 illustra un possibile schema di implementazione della unità che controlla la tastiera interagendo col bus di I/O).

La tecnica del Memory Mapped I/O richiede una fase di negoziazione fra dispositivo e sistema operativo (codice che di fatto si occupa delle interazioni con i dispositivi di ingresso uscita). Gli indirizzi riservati all'I/O sono definiti dal costruttore del processore. Come questi vengano mappati sugli indirizzi riconosciuti dai dispositivi come propri è normalmente oggetto di negoziazione durante l'inizializzazione dei dispositivi "plug-and-play".

E' da notare che gli indirizzi utilizzati per indirizzare i registri dei dispositivi di ingresso uscita potrebbero non essere soggetti al normale processo di traduzione da indirizzi logici a indirizzi fisici nella MMU o che la MMU stessa potrebbe implementare una traduzione dall'indirizzo riservato dal costruttore del processore a quello invece realmente utilizzato nel dispositivo².

Avendo visto come avviene l'arbitraggio delle risorse nella sezione 1.1.1, possiamo vedere più nel dettaglio come può avvenire la comunicazione relativa alla richiesta di esecuzione di una operazione su una periferica:

- il processore richiede una serie di store, agli indirizzi di I/O allocati nello spazio degli indirizzi di ingresso uscita riservati al dispositivo di I/O in oggetto. L'ultima store è quella relativa al segnale go
- per ciascuna delle STR R_n , $[R_n \dots]$:
 - la MMU riconosce un indirizzo di I/O
 - richiede l'accesso al BUS I/O all'arbitro
 - una volta ottenuto l'accesso
 - * scrive l'indirizzo, il dato da scrivere sulle linee del bus
 - * mette a 1 la linea $\overline{R}/W$
 - rilascia il bus

²questa cosa verrà chiarita quando affronteremo la parte sulla memoria virtuale e le relative tecniche per la traduzione da indirizzi logici, quelli generati dal processore, a fisici, quelli effettivamente utilizzati per l'indirizzamento della memoria fisica nonché di quella interna ai dispositivi di I/O secondo il modello memory mapped I/O

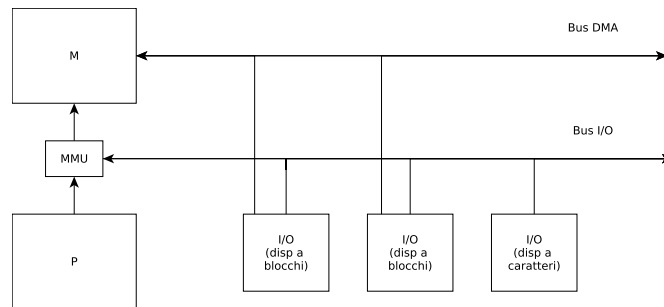


Figura 1.10: Sistema con periferiche in DMA

1.3 DMA

DMA sta per *direct memory access* ed è una tecnica che si usa per implementare i dispositivi di ingresso uscita in modo da permettergli un accesso controllato alla memoria centrale.

I dispositivi di I/O, per come li abbiamo visti nella Sez. 1.2 dovrebbero lavorare esclusivamente sulle loro memorie interne anche quando devono trattare operazioni che coinvolgono moli di dati relativamente grosse. Ad esempio, una unità di controllo di un disco lavora normalmente con operazioni su *blocchi*, ovvero mette a disposizione operazioni elementari di lettura e scrittura di un blocco del disco, di dimensioni dell'ordine del Kbyte. In questo caso, una ipotetica operazione di lettura richiederebbe che il processore trasferisse il blocco dalla memoria del dispositivo alla memoria centrale mediante una serie di load. Questo è chiaramente un problema, perché può impegnare il processore per un tempo relativamente lungo.

La tecnica del DMA consente ad un dispositivo di accedere direttamente alla memoria centrale del sistema. In pratica (vedi Fig. 1.10):

- il dispositivo è collegato alla memoria centrale mediante un *BUS DMA* e può effettuare letture e scritture in memoria quando ha ottenuto la possibilità di utilizzare il bus;
- il bus DMA è condiviso fra tutte le periferiche in grado di lavorare in DMA;
- il processore passa alle periferiche, quando ordina l'esecuzione di una certa operazione, gli eventuali indirizzi necessari a completare l'operazione direttamente in memoria centrale (questo avviene tramite il bus di I/O);
- la memoria diventa un dispositivo che deve essere in grado di soddisfare le richieste di lettura e scrittura che arrivano da più soggetti (processore, via interfaccia di memoria standard, e periferiche, via bus DMA) e deve pertanto essere dotato di una unità controllo più sofisticata.

Con queste assunzioni, un'operazione di lettura dal disco su un dispositivo che supporta sia Memory Mapped I/O che DMA avviene tramite i seguenti passi:

1. il sistema operativo (programma in esecuzione sul processore), invia alla periferica utilizzando il memory mapped I/O l'ordine di esecuzione della lettura che contiene:
 - la specifica che si tratta di una operazione di *lettura*;
 - l'*indirizzo del blocco* di disco interessato
 - l'*indirizzo in memoria* del buffer da utilizzare per i dati letti da disco;
2. il dispositivo legge i dati dal disco in un proprio buffer interno. Questa operazione deve avvenire in un buffer interno per ragioni di temporizzazione: attendere l'accesso al bus DMA e conseguentemente alla memoria centrale potrebbe risultare troppo lungo (o in ritardo) rispetto ai tempi dettati dal trasferimento dei dati dal dispositivo, una volta arrivati al blocco desiderato;
3. infine, il dispositivo acquisisce, interagendo con l'arbitro del bus, il controllo del bus DMA e avvia il trasferimento del blocco letto nella posizione di memoria il cui indirizzo base è stato passato dal sistema operativo fra i parametri dell'operazione di lettura.

Overhead del DMA Assumiamo che una CPU abbia un clock di 1GHz, sia dotata di un dispositivo disco con una banda di trasferimento di 4Mb/sec e che l'interfaccia di memoria sia in grado di trasferire 16b alla volta. Supponiamo anche che il trattamento delle interruzioni richieda 400 cicli, un trasferimento dati richieda 100 cicli e che l'inizializzazione del DMA richieda 1600 cicli e successivamente trasferisca blocchi da 16Kb alla volta. Infine, supponiamo che il processore occupi il 50% della banda di memoria.

Con questi dati, cerchiamo di capire quanto "costi", in termini di overhead sulle attività del processore, l'utilizzo del DMA.

Senza DMA, il processore spende

$$50\% \times (4Mb/sec)/(16b) * [(400 + 100)cicli/1000Mcicli/sec] = 0.625\%$$

Col DMA il processore spende

$$50\% \times (4Mb/s)/(16Kb) \times [(1600 + 400)/(1000Mcicli/sec)] = 0.025\%$$

1.3.1 Dispositivi DMA

I dispositivi che normalmente sono dotati di DMA sono quelli cosiddetti a *blocchi*, ovvero quelli che operano su blocchi di dato. Tipicamente, si tratta di dispositivi di memorizzazione di massa (dischi a stato solido e non), di rete

(interfacce di rete Ethernet), interfacce video, etc. Quelli che non sono dotati di DMA sono invece quelli cosiddetti *a caratteri* che comprendono dispositivi come tastiere, mouse, tavolette grafiche, etc.

1.3.2 Sull'utilizzo del DMA

Vanno fatte necessariamente un paio di osservazioni sull'utilizzo del DMA.

Primo punto: organizzazione della memoria. Senza DMA, il modello di utilizzo della memoria era molto semplice: il processore ordina operazione di lettura o scrittura, la memoria esegue. Adesso le operazioni possono essere richieste anche dai dispositivi in DMA. Dunque la memoria diventa una risorsa condivisa e necessita dell'introduzione di un arbitro per determinare, in caso di accessi concorrenti, se l'accesso vada garantito al processore o ai dispositivi di I/O. Di norma potremmo pensare che la priorità vada data al processore. In realtà sono i dispositivi ad avere spesso vincoli temporali più stretti. Un disco potrebbe avere la necessità di svuotare il buffer di lettura dalla memoria interna alla memoria del processore via DMA *prima* che il dispositivo si trovi nella condizione di poter leggere il prossimo blocco di dati, pena la perdita della lettura con conseguente attesa di un'altra rotazione del disco stesso per far sì che le testine ripassino sullo stesso settore. Inoltre, l'esistenza di una gerarchia di memoria pone il problema di dove vogliamo fare avvenire gli accessi DMA: di norma dovremmo pensare che vadano fatti nella memoria vera (non nelle cache ai vari livelli). Di fatto, in alcuni casi specifici potrebbe essere meglio poter scrivere i dati in DMA nella cache (L2 o L3) per esempio in caso di dati relativi a pacchetti di rete che devono essere processati immediatamente e rispediti. Un passaggio da M ritarderebbe troppo l'intera operazione.

Secondo punto: quali indirizzi? A questo punto del programma non abbiamo ancora visto il dettaglio della gestione della memoria virtuale con l'associato meccanismo di traduzione degli indirizzi da logici (processore) a fisici (memoria). Però abbiamo già visto più volte la necessità di un meccanismo che implementi un disaccoppiamento dello spazio di indirizzamento visto dal processore (spazio logico) da quello implementato nella memoria fisica disponibile (spazio fisico). Abbiamo anche detto che gli indirizzi logici verranno tradotti in fisici nella MMU che si trova fra il processore e la memoria. Ora il problema è: quando ordiniamo un'operazione a un dispositivo, da completare con il DMA, quali indirizzi passiamo per la memoria da accedere in DMA? Se passiamo indirizzi fisici, dobbiamo essere sicuri che a quegli indirizzi ci siano i buffer che effettivamente vogliamo utilizzare. Dobbiamo fare in modo che la gestione della memoria non "sposti" le pagine logiche in pagine fisiche diverse fra quando l'operazione viene richiesta e quando il DMA la implementa. Se passiamo indirizzo logici, il problema non sussiste, ma occorre che il dispositivo sia in grado di intergire correttamente con l'intero processo di traduzione di indirizzi da logici a fisici quando accederà alla memoria in DMA.

1.4 Interruzioni

Un'interruzione è un evento asincrono³ rispetto all'esecuzione del programma sul processore. Nel caso più generale, le interruzioni o eccezioni vengono utilizzate per scopi diversi:

- per gestire le operazioni di ingresso uscita; tenendo conto del fatto che il completamento di una operazione di I/O richiede tempi molto lunghi rispetto alla scala di tempi richiesta dal processore per eseguire le istruzioni assembler, si utilizzano le interruzioni per segnalare il completamento di una operazione di I/O, ordinata dal processore utilizzando memory mapped I/O. In questo modo, il processore è svincolato dalla necessità di attendere esplicitamente il completamento delle operazioni di ingresso uscita e può utilizzare il tempo speso dall'unità per completare l'operazione richiesta per svolgere altri compiti;
- per gestire situazioni di errore conseguenti ad azioni specifiche eseguite dal programma (fetch di una istruzione il cui codice non è riconosciuto come uno dei codici delle istruzioni assembler implementate dal processore, fault di pagina, divisione per 0).
- per eseguire chiamate di sistema, ovvero per chiamare parti di codice assembler con diritti diversi da quelli normalmente assegnati agli utenti (in generale con minimi livelli di privilegio);

Le interruzioni sono generate da dispositivi esterni al processore, quali dispositivi di I/O, ma anche dal sottosistema di memoria, per esempio in caso la memoria voglia segnalare un fault di pagina. Le eccezioni (interruzioni sincrone) sono invece quelle generate direttamente dal processore in caso di errori di vario genere.

Le interruzioni vengono *sentite* dal processore alla fine di ognuna delle iterazioni del ciclo *fetch-decode-execute*, come già accennato nelle parti precedenti del corso. Il processore, come tutte le unità firmware, implementa un ciclo simile al seguente:

```
1 while(true) {  
2   IR = fetch(PC);  
3   decode(IR);  
4   execute(IR);  
5   update(PC);  
6   if(interrupt) {  
7     interrupt_management();  
8   }  
9 }
```

³in caso di interruzioni legate ad errori che si possono verificare nel processore, sono in realtà eventi sincroni, diretta conseguenza di azioni intraprese durante l'esecuzione di istruzioni assembler da parte del processore. In questo caso sarebbe più corretto parlare di "eccezioni". Nel caso di architetture ARM il termine eccezione viene spesso utilizzato per tutti i casi, ovvero per le interruzioni (asincrone) e per le eccezioni vere e proprie (sincrone).

Ad ogni iterazione, in assenza di interruzioni, il processore preleva, decodifica ed esegue una singola istruzione assembler⁴.

Quando si verifica un'interruzione, il suo trattamento prevede una sequenza di passi standard:

- l'istruzione in esecuzione viene completata e lo stato del processore viene aggiornato di conseguenza (per esempio, viene calcolato il PC corrispondente alla prossima istruzione da eseguire e, se necessario, viene effettuato il writeback dei risultati nel file dei registri o nella memoria dati);
- si salva parte dello stato del processore (tipicamente almeno PC, LR e SP) e si procede ad eseguire una parte di codice assembler che
 - dipende dal tipo di interruzione che si è verificata, e
 - contiene l'intero codice necessario a gestire quel tipo di interruzione

L'esecuzione del codice di trattamento avviene in uno *stato* particolare, a seconda del tipo di interruzione, in generale con privilegi diversi (magiori) rispetto ai privilegi disponibili in nello stato “user” (stato in cui un normale utente del processore esegue i propri programmi);

- al termine, si ripristina lo stato del processore e, al prossimo ciclo **while(true)** si procede ad eseguire la “prossima” istruzione del programma che era stato interrotto, quella corrispondente al PC calcolato quando ci siano accorti dell'interruzione, prima di saltare alla routine di trattamento.

Se volessimo essere più precisi, il ciclo fetch-decode-execute presentato nel listato precedente andrebbe presentato in modo leggermente diverso, ovvero:

```

1 while(true) {
2     try {
3         IR = fetch(PC);
4         decode(IR);
5         execute(IR);
6         update(PC);
7     } catch (exception e) {
8         exception_management();
9     }
10    if(interrupt) {
11        interrupt_management();
12    }
13 }
```

Il listato evidenzia come, in presenza di eccezioni, venga immediatamente eseguito un handler. Se si è verificato un errore durante la *execute* (o la *fetch*), viene immediatamente invocato l'handler dell'eccezione, senza di fatto aggiornare il PC. Quando l'handler risolve il problema ritorna all'esecuzione del codice originale ripetendo l'istruzione che ha causato l'eccezione, visto che il PC non

⁴per non complicare la spiegazione stiamo assumendo di lavorare con un processore single o multi-cycle, non pipeline

Mode	Privileged	Purpose
User	No	Normal operating mode for most programs (tasks)
Fast Interrupt (FIQ)	Yes	Used to handle a high-priority (fast) interrupt
Interrupt (IRQ)	Yes	Used to handle a low-priority (normal) interrupt
Supervisor	Yes	Used when the processor is reset, and to handle the software interrupt instruction swi
Abort	Yes	Used to handle memory access violations
Undefined	Yes	Used to handle undefined or unimplemented instructions
System	Yes	Uses the same registers as User mode

Figura 1.11: Stati del processore ARM

User	System	Fast Interrupt	Interrupt	Supervisor	Abort	Undefined
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8_fiq	R8	R8	R8	R8
R9	R9	R9_fiq	R9	R9	R9	R9
R10	R10	R10_fiq	R10	R10	R10	R10
R11	R11	R11_fiq	R11	R11	R11	R11
R12	R12	R12_fiq	R12	R12	R12	R12
R13 (SP)	R13 (SP)	R13_fiq	R13_irq	R13_svc	R13_abt	R13_und
R14 (LR)	R14 (LR)	R14_fiq	R14_irq	R14_svc	R14_abt	R14_und
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)

Program Status Registers					
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_fiq	SPSR_irq	SPSR_svc	SPSR_abt
					SPSR_und

Figura 1.12: Registri duplicati (per modo operativo) in ARM

è stato ancora aggiornato.⁵ ARM riconosce un certo numero di stati diversi, riassunti nella Fig. 1.11. Gli stati Fast Interrupt e Interrupt sono gli stati che vengono utilizzati per il trattamento delle interruzioni generate dai dispositivi di ingresso uscita. Gli stati Abort e Undefined sono utilizzati per trattare le eccezioni generate da accessi illegali in memoria (per fetch o ldr/str) e per quelle generate da tentativo di esecuzione di una istruzione (assembler) illegale. Lo stato Supervisor viene utilizzato per l'esecuzione delle chiamate di sistema (istruzioni SVC), che sono anche loro gestite come "interruzioni software".

Ciascuno di questi stati dispone di alcuni registri duplicati. La Fig. 1.12 indica quali registri sono duplicati nei vari "modi" (stati) dei processori ARM. Vengono sempre messi a disposizione, nei vari modi che servono il trattamento delle interruzioni, registri duplicati per LR e SP. Questo significa che non occorre preoccuparsi di salvare, quando si passa ad uno di questi modi operativi, nessuno dei due registri. Questi registri vengono ripristinati quando si ritorna dal

⁵Questo meccanismo permette, per esempio, il trattamento del fault di pagina. La MMU genera un'eccezione, l'handler provoca l'esecuzione della parte di sistema operativo che si occupa del page fault. Quando il trattamento del page fault da parte del sistema operativo è terminato, il programma che aveva generato il fault viene fatto ripartire e riparte esattamente dal punto in cui si era fermato, cioè ordinando lo stesso accesso in memoria che aveva causato il fault. Questa volta l'accesso avverrà senza problemi visto che il sistema operativo ha appena effettuato i passi necessari a portare in memoria centrale la pagina mancante.

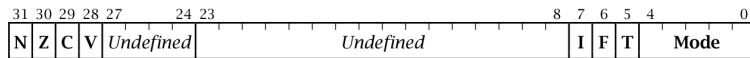


Figura 1.13: Contenuto del registro CPSR

trattamento dell'interruzione che provoca il cambiamento di stato. Inoltre, tutti i modi mettono a disposizione un registro SPSR (Saved Program Status Register) che mantiene lo stato del CPSR al momento del passaggio di stato in modo che tale registro possa essere ripristinato al rientro dal trattamento dell'interruzione. Il modo Fast Interrupt mette a disposizione una copia anche dei registri general purpose da R8 a R12. Questo modo è stato previsto per quelle interruzioni che richiedono un trattamento molto breve e che necessita di pochi registri general purpose. Nel caso ne bastino 5, possiamo utilizzare durante il trattamento di una fast interrupt i registri R8–R12 senza bisogno di salvarli sullo stack: nel modo fast interrupt infatti usiamo le copie (grigie nella figura) e non i registri originali.

In una gran parte dei casi, il codice eseguito per trattare un'interruzione è eseguito *ad interruzioni disabilitate*, ovvero facendo in modo che ulteriori interruzioni non possano essere prese in considerazione se non dopo che il trattamento dell'interruzione corrente sia effettivamente terminato.

Questo effetto si ottiene, in ARM, intervenendo sul registro CPSR (Current Program Status Register) che contiene due bit (I ed F) che, se messi a 1, rispettivamente mascherano (fanno in modo che non vengono trattate) le interruzioni normali e quelle “fast” (vedi oltre). Ci sono anche una serie di casi in cui un'interruzione può essere a sua volta interrotta da una interruzione il cui trattamento sia più urgente e semplicemente possa essere completato in modo molto veloce senza disturbare il trattamento dell'interruzione corrente. In questo caso occorre includere nel codice per il trattamento delle interruzioni “interrompibili” tecniche di programmazione che assicurano che non vengono persi (sovrascritti) dati in caso di interruzioni di interruzioni⁶.

Le interruzioni vengono segnalate al processore utilizzando appositi pin del processore che portano all'unità controllo 1 o più bit che vengono utilizzati, quando sono messi a 1, per segnalare tipi diversi di interruzioni. Arm per esempio, ha a disposizione, fra gli altri, due segnali diversi per segnalare un'interruzione “normale” (bit IRQ) oppure un'interruzione “fast” (bit FIQ). Le interruzioni di tipo fast vengono utilizzate per segnalare eventi che possono essere trattati molto rapidamente. Quelle di tipo normale richiedono tempi di trattamento più lunghi. Normalmente, a ciascuno dei due tipi di interruzioni possono corrispondere interruzioni generate da dispositivi diversi. Per esempio, dischi e schede di rete genereranno entrambe un'interruzione IRQ. Hardware dedicato provvederà a presentare al processore un unico segnale (da un bit) IRQ

⁶un po' come quando si programmano routing ricorsive e si utilizza lo stack invece che i registri per il passaggio dei parametri per fare in modo che siano supportate le chiamate ricorsive alla funzione

0	reset
4	undef instruction
8	sw interrupt (SVC)
12	abort (fecth da indirizzo illegale)
16	abort (ldr/str indirizzo illegale)
20	reserved
24	IRQ
28	FIQ

Figura 1.14: Tipo di interruzioni ARM

risultato dell'OR di tutti gli IRQ delle diverse unità di controllo delle periferiche e messo in AND con la negazione del bit I del registro CPSR. Il motivo dell'interruzione è di norma posto in un indirizzo particolare di memoria, noto alla routine che gestisce le interruzioni, che lo può quindi controllare per capire quale dei dispositivi ha veramente messo IRQ a 1.

In particolare, quando si verifica un'interruzione il processore, una volta completata l'esecuzione dell'istruzione corrente ed aggiornato lo stato del processore esegue una serie ben precisa di passi⁷:

- salva il valore del program counter corrente (è l'indirizzo di ritorno dall'interruzione, di fatto) nella copia del registro LR disponibile in ognuno dei modi operativi
- salva il CPSR nel SPSR
- setta il modo (stato) del processore nella CPSR (vedi Fig. 1.15). Per accedere le diverse parti del registro di stato vengono utilizzate due istruzioni particolari: la MRS (move status to register) che permette di copiare il contenuto del CPSR in un registro generale (per esempio, MRS R0, CPSR lo copia nel registro generale R0) in modo da poter successivamente manipolarne il contenuto (testare o settare/azzerare particolari bit o configurazioni di bit) e l'istruzione duale MSR (move register to status) che scrive un registro generale nel CPSR⁸ (per esempio, MSR CPSR, R0 copia il contenuto di R0 nel CPSR). In certi casi, sono disponibili modi per indicare flag particolari del CPSR come destinazioni delle MRS. Per esempio, MSR CPSR_F, #1 setta il bit di mascheramento delle fast interrupt.
- setta il bit T della CPSR a 0 in modo che il processore esegua istruzioni normali anziché Thumb

⁷l'elenco che segue è sempre riferito all'architettura ARM, architetture differenti potrebbero eseguire azioni leggermente diverse o utilizzare indirizzi diversi

⁸versioni diverse del processore hanno modalità leggermente diverse di utilizzo di queste due istruzioni, per esempio nel modo utilizzato per nominare i registri di stato (CPSR, APSR, ...). Queste istruzioni potrebbero essere disponibili solo in certi "modi", a seconda della classe di processori ARM utilizzati

Mode Bits		Processor Mode (Abbreviation)	Accessible Registers
Bin	Hex		
10000	10	User (usr)	PC, R14-R0, CPSR
10001	11	Fast Interrupt (fiq)	PC, R14_fiq-R8_fiq, R7-R0, CPSR, SPSR_fiq
10010	12	Interrupt (irq)	PC, R14_irq, R13_irq, R12-R0, CPSR, SPSR_irq
10011	13	Supervisor (svc)	PC, R14_svc, R13_svc, R12-R0, CPSR, SPSR_svc
10111	17	Abort (abt)	PC, R14_abt, R13_abt, R12-R0, CPSR, SPSR_abt
11011	1B	Undefined (und)	PC, R14_und, R13_und, R12-R0, CPSR, SPSR_und
11111	1F	System (sys)	PC, R14-R0, CPSR

Figura 1.15: Modi ARM (rappresentazione nel CPSR)

- a seconda dei casi, disabilita interruzioni normali e/o fast settando i bit F e I del registro CPSR
- salta ad eseguire il codice che si trova all'indirizzo `0x0000000 + tipo dell'interruzione`, dove il tipo dell'interruzione è definito come in Tab. 1.14.

Dualmente, il ritorno dal trattamento interruzioni esegue le seguenti azioni:

- sposta il contenuto del registro LR in PC⁹;
- copia il registro SPSR in CPSR, cosa che ripristina il modo di funzionamento precedente all'interruzione, e
- azzerà i bit di mascheramento delle istruzioni, in caso fossero stati settati a 1 (questi sono i bit I e F della parola di stato in CPSR).

Vediamo, per chiarire i concetti, come avviene il trattamento di una sw interrupt (il tipo di interruzione (eccezione, in verità) causato dall'esecuzione di una istruzione SVC 0).

L'istruzione SVC è rappresentata in memoria da 8 bit con il codice corrispondente all'istruzione e 24 bit utilizzati per rappresentare il parametro (imm24):

[COND|1111| 24 bit immediate]

L'esecuzione della SVC comporta i passi che abbiamo dettagliato poco sopra, al termine dei quali si salta ad eseguire l'istruzione all'indirizzo `0x00000008`. Questa istruzione dovrebbe saltare immediatamente un pezzetto di codice simile al seguente:

```

1  LDR R1, [LR, -4]           ; carica in R1 la svc
2  BIC R1, #0xff000000       ; cancella 8 bit piu' significativa
3  LDR R0, =svcvec          ; base vettore con indirizzi svc[i]
4  LDR PC, [R0, R1, LSL #2] ; salto alla routine svc[i]
```

⁹in alcuni casi dopo averci sommato un piccolo offset

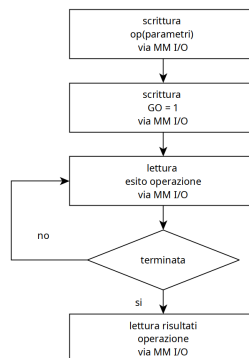


Figura 1.16: Programmed I/O

1.5 Utilizzo di un dispositivo di I/O

In questa sezione, vediamo come i meccanismi che abbiamo visto nelle sezioni precedenti possono essere utilizzati per implementare diverse modalità di interazione con i dispositivi di I/O.

1.5.1 Memory mapped I/O e polling

In questa modalità (vedi Fig. 1.16) le operazioni vengono comandate dal processore con *STR* nella memoria interna dell'unità di I/O. La terminazione dell'operazione è testata sempre dal processore con *LDR* ripetute che leggono la parola di stato che segnala il completamento dell'operazione corrente nella memoria interna dell'unità di I/O. Il processore esegue anche eventuali altre operazioni di *LDR* per recuperare i risultati dell'operazione di ingresso uscita (per esempio i dati letti dal dispositivo) e memorizzarli in memoria centrale. E' quello che abbiamo già considerato all'inizio della Sea. 1.2.

Overhead del polling Supponiamo di avere un processore con clock a 1Ghz e che un singolo ciclo di polling costi 400 cicli di clock.

Se dovessimo controllare una periferica tipo mouse, che è una periferica “lenta”, potremmo desiderare di controllare se la posizione del mouse o lo stato dei bottoni fosse cambiato 30 volte al secondo. Questo richiederebbe

$$30 * 400 = 12000 \text{ cicli/s}$$

La percentuale di cicli spesi per controllare il mouse sarebbe dunque

$$(12K \text{ cicli/s}) / (1000M \text{ cicli/sec}) = 0.0012\%$$

che rappresenta un overhead tollerabile.

Ma se invece dovessimo controllare l'attività di una periferica “veloce” come ad esempio un disco con trasferimenti di 4Mb/s e un'interfaccia capace di trasferire 16b alla volta? Per non perdere dati, dovremmo eseguire almeno 4Mb/s / 16b polling al secondo, ovvero 250K polling al secondo. Ogni polling costerebbe

$$250K/s \times 400 = 100M \text{ cicli/s}$$

per cui la percentuale dei cicli di polling sul totale salirebbe al

$$100M / 1000M = 10\%$$

Dal momento che questo è un tempo necessario per il solo test dei registri (non è il tempo impiegato per il trasferimento vero e proprio dei dati da/per la memoria principale), la percentuale è chiaramente troppo alta.

1.5.2 Interrupt-driven I/O

In questa modalità (vedi Fig. 1.17) le operazioni vengono comandate come nel caso precedente utilizzando memory mapped I/O. Tuttavia, una volta che l'operazione è stata comandata, il processore “sospende” il codice che ha ordinato l'operazione e passa ad eseguire altro codice. In un sistema operativo multiprocessing, tipicamente il processo che ha ordinato l'operazione di ingresso uscita viene messo in stato di “attesa” e viene mandato in esecuzione un altro processo dalla lista dei processi “pronti” per l'esecuzione.

La fine dell'operazione di ingresso uscita viene segnalata dal dispositivo mediante il meccanismo delle interruzioni. Il dispositivo che ha completato l'operazione segnala un'interruzione al processore. Il processore, alla fine dell'esecuzione dell'iterazione corrente del suo ciclo *fetch-decode-execute* testa il segnale di interruzione e rileva che è alto. A questo punto il PC passa a puntare (in modalità “kernel”) alla prima istruzione del trattamento delle interruzioni. Questa si trova in una posizione fissa e nota nella parte bassa dello spazio di indirizzamento dei processori ARM. L'istruzione è un salto alla routine di trattamento delle interruzioni che, in breve, capisce chi ha sollevato l'interruzione e perchè e risveglia il processo che aveva chiesto l'operazione di ingresso uscita che è appena terminata. Alla fine della routine di trattamento delle interruzioni,

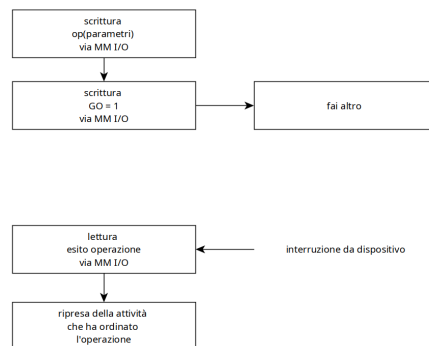


Figura 1.17: Interrupt-driven I/O

viene ripresa l'esecuzione delle istruzioni dal PC che era stato aggiornato nel ciclo *fetch-decode-execute* immediatamente prima della verifica dell'esistenza dell'interruzione.

Quando il processo che aveva ordinato l'esecuzione dell'operazione di ingresso uscita riprende l'esecuzione, può svolgere le operazioni necessarie per terminare l'operazione di ingresso uscita:

- se l'esito dell'operazione e gli eventuali dati risultato fossero ancora nella memoria interna del dispositivo di ingresso uscita può avviare un ciclo di letture che, utilizzando il memory mapped I/O, trasferiscono queste informazioni nella memoria principale; successivamente può testare l'esito dell'operazione di I/O e proseguire;
- se invece il dispositivo ha già trasferito tali informazioni nella memoria principale mediante DMA, può semplicemente testare l'esito dell'operazione di I/O e proseguire.

Overhead interrupt driven I/O Con le stesse assunzioni precedenti (processore con clock a 1Ghz e trattamento dell'interruzione che occupa 400 cicli, proviamo a vedere quanto tempo impieghiamo nel trattamento delle interruzioni per le operazioni su disco.

Con disco che trasferisce 4Mb/s in blocchi da 16b, il tempo per i trasferimenti sarebbe

$$(4Mb/s)/(16b) \times [400cicli/1000Mcicli/s] = 10\%$$

E' lo stesso valore ottenuto con il polling ma in questo caso il tempo è speso dal dispositivo mentre il processore fa altro (esegue istruzioni di altri processi).

1.6 Ingresso uscita: divisione delle responsabilità

La responsabilità della gestione delle attività legate all'ingresso/uscita è interamente del Sistema operativo. I modi di protezione del processore garantiscono che in modalità utente non sia accessibile nessuno dei meccanismi con cui si interagisce con le unità di ingresso uscita (memory mapped I/O, che vuol dire indirizzi riservati all'ingresso/uscita, interruzioni, che vuol dire routine di trattamento delle interruzione, etc.).

Come facciamo dunque ad utilizzare le routines che interagiscono con l'ingresso uscita da programma? Per esempio, nel laboratorio abbiamo visto una chiamata alla funzione `fread` che, in caso il descrittore `FILE *` utilizzato come parametro derivi da una `fopen` di un file su disco, richiederà di effettuare delle letture sul dispositivo di ingresso uscita.

La sequenza di eventi generati dalla `fopen` è questa:

1. il codice del vostro main chiama la `fopen` il cui codice è nella `libc`.
2. nel codice della `libc`, eseguito in modalità utente, si invoca una chiamata di sistema `read`. L'invocazione avviene mediante una `SVC 0` che comporta il passaggio alla modalità *kernel*
3. l'implementazione della `read` capisce che si tratta di interagire con il disco¹⁰ e chiama le routine del `driver` del disco per effettuare l'operazione. Il `driver` contiene sia il codice che ordina l'operazione al dispositivo via memory mapped I/O che la routine di trattamento dell'interruzione che servirà a processare le interruzioni di fine operazione dell'unità disco
4. la routine del `driver` che ordina il comando al disco termina, viene de-schedulato (messo in attesa) il processo che aveva chiamato la `fread` e mandato in esecuzione un altro processo "pronto".

Visto che il disco è un dispositivo "a blocchi", quando il dispositivo termina l'operazione di lettura il dato letto sarà già stato trasferito nel buffer specificato dalle `fread` via DMA. Dunque:

1. il dispositivo segnerà una interruzione
2. il processore eseguirà l'handler per quel tipo di interruzione
3. che verificherà che l'operazione è terminata senza errori, e
4. farà in modo che il processo che ha chiamato la `fread` venga rimesso nella lista dei pronti

¹⁰omettiamo qui il fatto che serve consultare il File System, altra parte del sistema operativo, per capire quale porzione di disco vada effettivamente letta

5. il processore riprenderà l'esecuzione della sequenza di istruzioni interrotte dal valore del PC che era stato aggiornato come ultimo passo del ciclo fetch decode execute prima di passare al controllo delle interruzioni.

Quando il processo che aveva ordinato la `fread` otterrà nuovamente l'utilizzo del processore, continuerà la propria esecuzione avendo nel buffer della `fread` i dati letti dal disco.